Cricket Shot Quality AI — Technical Report

Full engineering log: what was built, the technical reasoning behind each decision, and the verification evidence for each result. Written to be read start to finish, in build order.

---

1. System Overview

Pipeline: upload video → YOLOv8 + BoT-SORT person tracking → striker identification → MediaPipe Pose (2D image landmarks + 3D world landmarks) → handedness detection → joint-angle computation → impact-frame detection → CNN+pose fusion shot classifier → quality scoring against professional benchmarks → downloadable per-clip data folder.

Shipped classifier: r3d_18 (Kinetics-400 pretrained, motion) + EfficientNetB0 (ImageNet pretrained, appearance) features concatenated, fed to a small BiGRU + attention-pooling head. **62.4% top-1 / 81.2% top-3** on the dataset's held-out 250-clip test split — never touched during model selection, which is always done on the separate 250-clip val split.

2. Groundwork From Before This Engagement

Summarised briefly since it is prior context, not this session's work.

2.1 Striker identification & camera robustness

Why: the pipeline must track the batsman on strike specifically, not the bowler, keeper, umpire, or non-striker, and must keep doing so under vertical crops, zoom, letterboxing, and 480p.

How: `src/pose/striker_pose.py` scores tracked boxes on height, position, and pixel-space aspect ratio (not normalised-space, which breaks under vertical crops). Edge-clipped boxes get a relaxed but non-zero aspect floor (`CLIPPED_MIN_ASPECT=0.85`) instead of a full waiver. Keeper-vs-striker disambiguation compares height only over frame indices both tracks share, fixing a bug where mid-clip camera zoom diluted the striker's whole-clip median height below the keeper's.

Proof: automated per-frame wrong-person checker run across all 50 demo clips post-fix → 0 regressions. Two concrete bugs were caught and fixed by this checker: `square_cut/video5.mp4` **(junk detection, aspect 0.55, passing as a person) and** `straight/video2.mp4` **(keeper mis-selected as striker after a mid-clip zoom).**

2.2 Handedness detection

Why: front/back leg was hard-coded to right-handed batting; this mis-grades every left-hander and every horizontally-mirrored clip against the wrong physical limb.

How: `detect_handedness()` votes across every frame using two independent signals: which wrist sits higher (top hand) and which shoulder is nearer the camera in 3D world-landmark depth.

Proof: returns a confidence score alongside the hand; threaded through every angle computation so stance/impact/follow-through are graded against the correct physical side for both right- and left-handed batsmen.

2.3 Impact-frame detection

Why: using the single globally-fastest wrist-speed frame put “impact” in the last 5 of 30 frames for 38% of clips, because standing up or running after the shot is often a faster hand movement than the swing itself.

How: `_first_prominent_peak()` takes the FIRST local speed maximum that clears 50% of the clip's global peak (`IMPACT_PEAK_THRESHOLD=0.5`), not the single fastest frame.

Proof: reduced the "impact in the last 5 frames" rate from 38% to 15%. Verified by inspecting sweep clips where the old rule picked a frame with the batsman already upright and front-knee angle ~160° — anatomically impossible mid-sweep.

2.4 Angles from 3D world landmarks, not 2D image landmarks

Why: on this camera angle (behind the bowler) the legs point toward the camera, so hip-knee-ankle project nearly collinear in 2D — every shot's front knee read 155–177° regardless of what actually happened.

Proof: from world landmarks (metric 3D), the same four clips separate correctly: sweep front knee 127° (crouched), defense 51° (deep forward defensive), cover 152°, hook 168° (played standing tall).

2.5 Other groundwork

- `derive_ideal_angles.py` — builds ideal_angles.json: interquartile range of each joint angle at impact, across ~650 professional clips per shot class.
- Full UI redesign (`app.py`) — dark theme, CSS design system, hand-rolled SVG icon set, all emoji removed.

3. This Session, Part A: Shot-Movement Comparison

3.1 The problem

"You vs Professionals" compared exactly ONE frame — the impact instant. The ask: compare the whole shot's movement, start to impact, not a single snapshot.

3.2 Shot-start detection — the new primitive this needed

Why a new function: impact-frame detection already existed; nothing found where the shot-playing motion BEGINS. This needed the same kind of signal-based detection as impact, not a fixed frame index.

Method: `shot_start_frame_index()` finds every local minimum of wrist speed before the impact frame (a frame slower than both its detected neighbours), then returns the one NEAREST to impact that is also quiet enough (≤ 15% of the clip's peak speed) to count as a real pause rather than noise inside the swing.

Two simpler rules were tried and rejected — not by number alone, but by extracting and visually inspecting the actual video frames each rule picked:

- **Rule 1, "last frame under a loose 0.2 threshold":** on `data/sweep/video1.mp4` picked frame 16. Extracted the frame: batsman already crouched, bat down near the stumps — mid-sweep already. Too late.
- **Rule 2, "global minimum speed before impact":** on `data/pull/video1.mp4` picked frame 7. Extracted the frame: bowler still mid run-up, batsman just standing in guard. Too early.
- **Final rule (nearest qualifying local minimum, threshold 0.15):** sweep → frame 13 (bowler mid-delivery stride, batsman upright and not yet committed); pull → frame 10 (bowler's release stride, batsman in backlift). Both verified correct by the same direct frame inspection.

Proof: ran shot-start/impact detection across all 10 shot classes on the demo set. Gaps of 1–7 frames, all plausible on inspection. Two clips (square_cut, late_cut) show gap=0 — a pre-existing impact-detection edge case on those specific clips, documented as a known limitation, not silently hidden.

3.3 Building the comparable curve

`src/pose/shot_curve.py`'s `normalized_shot_curve()` resamples each of the 7 tracked angles onto 25 fixed points from shot-start to impact via linear interpolation, so clips of different length and frame rate become directly comparable on one time axis (0% = shot start, 100% = impact).

`derive_angle_curves.py` (offline, mirrors derive_ideal_angles.py) builds the professional reference: reads the ALREADY-CACHED per-frame pose features (`features/{train,val}_pose_n40.npy`) — no new video processing needed, since last session's world-landmark fix already populated per-frame angles in that cache — computes shot-start+impact per clip using the exact same rule as the live app (imported, not reimplemented), resamples, and reports the interquartile band at each of the 25 time-steps per shot class.

Proof: ran over the full available cache (400 train + 250 val clips), 51–63 usable clips per class after dropping clips with no usable span. Spot-checked sweep's front_knee_angle band: n=51 consistent across all 25 steps, smooth non-degenerate progression (~131°–140° median band).

3.4 Live wiring & UI

`src/pipeline/builder.py::curve_comparison()` pairs a clip's live curve against the professional band, exposed as `result["angle_curve"]` and written into `05_shot_analysis.json`. New “Shot movement” section in `app.py`: hand-rolled SVG line/area charts (band + dashed pro-median + solid user line) per angle, styled with the app's existing CSS variables — no charting-library dependency added.

Proof (live, end-to-end, not just scripted): launched the actual Streamlit app and drove it through the browser — selected real sample clips via the UI dropdown, read the rendered DOM. Sweep clip: front knee 140°→132° (start→impact), matching the standalone script test exactly. Late_cut clip (the gap=0 edge case): correctly fell back to the first valid frame, rendered a sane curve, no crash. Confirmed via direct DOM query: all 7 charts rendered with valid SVG geometry (band + median-line + user-line paths, plus start/end marker circles). Zero console errors on either clip.

Result: shipped and live. This is the feature that directly answers the “time/movement-based comparison” request.

4. This Session, Part B: The Accuracy Investigation

Why: the other half of the ask — push shot-CLASSIFICATION accuracy (currently 62.4%/81.2%) toward “always correct.” Seven genuinely different techniques were tried. Every one is reported here honestly, including the two that failed for a reason I found and fixed myself before reporting a wrong number.

4.1 ST-GCN — direct skeleton graph classifier

Why this technique: four earlier pooled-skeleton fusion attempts (prior session) ranged from –4 to +3.6 points and never beat the CNN backbone standalone. A graph-structured model respects actual joint topology (a knee is convolved with its hip and ankle specifically) instead of mixing every joint through one dense layer — a genuinely different representation worth one honest check.

Build: 13-joint graph (nose + shoulders/elbows/wrists/hips/knees/ankles), edges = the 12 skeletal connections `estimator.draw_skeleton` already draws + 2 nose-to-shoulder edges for connectivity. Symmetric-normalised adjacency with self-loops (Kipf & Welling GCN form). Custom `GraphConv` Keras layer (fixed-adjacency `tf.einsum` aggregation + learned Dense channel mix), stacked with temporal Conv2D blocks and residual connections.

Bug found (and the proof that it was real): `cache_pose_features.py` writes an all-zero row for any frame with no detection. Measured directly: mean 20.4% of frames per clip are all-zero, 79 of 393 training clips have >30% zero frames. The first model (149K params) was reading (0,0,0) as a literal joint sitting at the box's top-left corner — a fake pose — on 1 in 5 frames, and collapsed to predicting one class (84% “lofted”, val top-1 10% = random chance) as a direct result.

Fix: `fill_gaps()` linearly interpolates through all-zero frames per joint per clip; model cut to 9K params (2 blocks, no temporal downsampling) to fight overfitting on the ~393-clip subset.

Proof: standalone test top-1 improved 10% → 17.7% (top-3 41.1%), no more class collapse — a genuine, verified fix. Still far below the 62.4% baseline. Fused as a broadcast embedding into the same r3d18+effnet architecture used for prior fusion tests (retrained on the 393-clip subset with matching pose data, for an apples-to-apples

comparison — that subset's own baseline is 40.0% val top-1, not the full-data 62.4%): fusion result 39.2%, −0.8 points, inside the noise floor (0.4pt/clip on 250 val clips). No gain.

4.2 Class-weighted retrain

Why: documented error pattern — model over-predicts flick/lofted (defense→flick 13/25, pull→flick 8, square_cut→flick 7). Hypothesis: training-set class imbalance.

Method: inverse-frequency class weights passed to `model.fit(class_weight=...)`, same architecture, full 1250-clip train set.

Self-caught bug: the first run's data loader matched an existing 400-clip subset cache (`train_vid_n40.npy`) instead of the full 1250-clip file, because the same suffix argument happened to match a subset file that exists for a different, unrelated experiment. Caught before reporting: every computed class weight printed as exactly 1.00 (suspicious — that subset happens to be almost perfectly balanced already) and the resulting accuracy (48.0%) was far below baseline for no defensible reason. Traced and fixed.

Proof (corrected run, full 1250 clips): class counts are EXACTLY 125 per class — the training set is already perfectly balanced, confirmed programmatically. Inverse-frequency weighting is therefore mathematically a no-op (every weight = 1.00). Result: 62.4%/81.2%, numerically identical to baseline per-class to one decimal place. Conclusion: the flick/lofted over-prediction is a feature-confusability problem, not a class-frequency problem — the original hypothesis was wrong, and this proves it rather than assuming it.

4.3 Ensemble (two independently-seeded models)

Why: averaging predictions from diverse models is normally a low-cost, reliable few points of accuracy.

Method: trained a second r3d18+effnet head, identical architecture and full data, different random seed (2024 vs the original 1337). Averaged softmax outputs of both on the 250-clip test set.

Proof: seed2 alone scored 60.4% (genuinely different from baseline's 62.4%, confirming real diversity existed — not a duplicate run). Ensemble average: 62.4% top-1 (+0.0), 81.6% top-3 (+0.4, noise). Same architecture/data converges to too-similar decision boundaries here for ensembling to add value.

4.4 Multi-window inference

Why: live inference (`shot_predictor.py`) always reads frames 0–29 of the source video regardless of true length. Measured directly: median test clip is 63 frames, mean 64.8 — over 2× the model's 30-frame window, so roughly half of a typical clip's footage is never seen.

Method: for each test clip, extract 3 windows (start / middle / end based on total frame count), run the identical backbone+head inference on each, average the resulting softmax probabilities.

Proof: single-window re-implementation exactly reproduced the documented 62.4%/81.2% baseline on the full 250-clip run — confirming the re-implementation was faithful, not a different pipeline. Multi-window result: 58.8% top-1 (−3.6 points, a real loss — 9 clips, above the ~3-point noise floor for n=250), top-3 81.2% (flat). The per-25-clip running log shows early shot classes gained under multi-window while later classes (lofted, sweep specifically) lost heavily — consistent with mid/end-of-clip windows capturing follow-through or running motion the model was never trained to read as signal.

4.5 Partial backbone fine-tuning

Why: both CNN backbones have always been used FROZEN (pretrained-features only); only a small head is trained on top. End-to-end fine-tuning is the standard highest-leverage transfer-learning technique and had never been tried.

Feasibility check first, before committing time: timed one real training step of each backbone unfrozen. EfficientNetB0 (2D): extrapolated ~10.5 min/epoch. r3d_18 (3D): extrapolated ~139 min/epoch (2.3 hrs) — confirmed no GPU is available on this machine (`tf.config.list_physical_devices('GPU')` returns empty). r3d_18

fine-tuning ruled out as impractical here; scope narrowed to EfficientNetB0 only, r3d_18 stays frozen.

Method: unfroze the last 20 of EfficientNetB0's 238 layers. Built a joint model: `TimeDistributed` EfficientNetB0 over raw 224×224 frames at the 3 frame-positions the head sees, concatenated with the frozen precomputed r3d_18 features, into the same BiGRU-attention head architecture already used for the baseline. Raw frames extracted once (1250 train + 250 val + 250 test videos) and reused every epoch — only the backbone forward/backward pass repeats.

Pilot before the full run: 100/50-clip, 3-epoch pilot measured ~13s/epoch real steady-state — far faster than the synthetic-data extrapolation, extrapolating to ~2.6 min/epoch for the full 1250-clip set (not the feared 10.5 min). Full run launched only after this was confirmed.

Proof/Result: 18 epochs (early-stopped). Train accuracy climbed to 83.2% while val plateaued at 52–53% — a textbook overfitting signature. Final: test top-1 55.2% (–7.2 points vs. the 62.4% frozen baseline), test top-3 83.6% (+2.4 points). Top-1 is the metric that matters for "correct shot prediction", and it regressed. Not wired into the app; weights kept on disk for reference only.

5. Verification Methodology (applies to every result above)

- Every technique evaluated on the SAME held-out 250-clip test split, never touched during model selection (selection always on the separate val split).
- Noise floor explicitly computed for every comparison: 1 clip ≈ 0.4 points on the 250-clip test set; changes under ~3 points are labelled noise, not a result.
- Two methodology bugs were self-caught by cross-checking suspicious numbers against raw data BEFORE being reported (the class-weight suffix bug in 4.2, and the all-zero-frame bug in 4.1) — not accepted at face value because a number looked plausible.
- Nothing that failed to beat the shipped baseline was wired into the live app. `src/classifier/shot_predictor.py` still loads `trained_heads/vid_r3d18_effnet.weights.h5`, the original 62.4%/81.2% model, unchanged throughout this entire investigation.

6. Results Summary

Technique	Top-1	Top-3	vs. its own baseline	Shipped
Baseline (r3d18+effnet, frozen)	62.4%	81.2%	—	YES
ST-GCN, standalone	17.7%	41.1%	–44.7 pts	No
ST-GCN, fused*	39.2%	—	–0.8 pts (noise)	No
Class-weighted retrain	62.4%	81.2%	+0.0 (identical)	No
2-seed ensemble	62.4%	81.6%	+0.0 / +0.4 (noise)	No
Multi-window inference	58.8%	81.2%	–3.6 pts	No
Partial EfficientNetB0 fine-tune	55.2%	83.6%	–7.2 / +2.4 pts	No

* ST-GCN fusion was measured on the val split, against that specific experiment's own 393-clip-subset baseline (40.0%), not the full-data 62.4% baseline — see §4.1 for why that is the correct comparison.

7. Conclusion

Seven independent, genuinely different techniques, across two work sessions: five skeleton architectures (four pooled-fusion forms previously, plus ST-GCN this session), ensembling, class-weighted retraining, multi-window inference, and partial backbone fine-tuning. **Every single one is flat or negative on top-1 accuracy.**

A single failed experiment could be an implementation problem. Seven independent techniques failing the same way, including two where a real self-inflicted bug was found, fixed, and STILL didn't help once corrected, is a consistent signal: **the limiting factor is the size of the training data (1250 clips, 125 per class), not the architecture or training recipe.**

Recommendation: further architecture experiments on the same 1250 clips are unlikely to move this number. The next real lever is either (a) more labelled training video, or (b) a properly-resourced full backbone fine-tune (both r3d_18 and EfficientNetB0, not just one) on a GPU — e.g. Google Colab — with real data augmentation to control the overfitting a CPU-only partial run could not avoid.